

边轩凯



意向岗位：AI Agent 开发实习生 | Java 后端方向 | 可长期实习

电话：17773923713

邮箱：bxk0862004@163.com

教育经历

2023.09-2027.06

内蒙古民族大学

软件工程

统招本科

- **学业表现**: 专业排名前 1%，连续三年获得校级一等奖学金，获得校企合作一等奖学金
- **主修课程**: C++、Java、计算机网络、操作系统原理、数据结构与算法、计算机组成原理。

专业技能

- 1.熟悉 RAG 全链路设计与优化，能够结合业务场景对召回效果、响应质量与系统性能进行针对性优化。
- 2.熟悉 Agent 基本原理，了解任务规划、工具调用、流程编排等核心机制，掌握 Agent 记忆系统的基础设计思路。
- 3.熟悉 Claude Code、OpenClaw、Codex 等 Agent 的使用，了解架构设计原理，能够结合实际场景进行使用与实践。
- 4.熟悉深度学习框架 pytorch，并针对具体业务做出针对性优化。
- 5.熟练掌握 LangChain4j 框架，熟悉 LangGraph4j 的基本用法；具备 AI 工作流编排、工具调用、代码生成类应用的实现经验
- 6.掌握 Java 基础（集合、多线程、JVM），熟悉 Spring Boot、Spring Cloud 微服务。
- 7.掌握 Docker 容器化部署、Nginx 反向代理配置，了解 K8s 基础使用，可独立完成项目部署运维。

项目经验

2025.12-2026.02

多模态企业智能助手平台（Agent + RAG 混合架构）

核心开发者

项目背景: 企业内部存在文档查询、系统操作相互脱节的问题，日常办公效率偏低。我参与开发这款结合 Agent 与 RAG 的智能助手平台，主要负责 RAG 检索优化、多模态文档处理、Agent 基础引擎开发，实现文档智能问答和简单任务自动执行，辅助员工日常办公

技术栈: Spring Boot 3 / Spring Cloud Alibaba、Nacos、OpenFeign、Redis、RabbitMQ、LangChain4j、Milvus、Elasticsearch、通义千问/DeepSeek、PyTorch、Claude Code、JMeter、Docker、K8s

工作内容:

- 1.基于 LangChain4j 搭建 Agent 基础能力，实现 ReAct 模式的任务拆分与循环判断逻辑，搭建简易工具注册中心，对接多个内部业务接口，让 Agent 可以执行常规基础任务。
- 2.参与搭建 Milvus 向量检索 + Elasticsearch 关键词检索的双路召回架构，使用 PyTorch 对 BERT 重排序模型做微调，逐步优化检索匹配效果。
- 3.接入 PaddleOCR 和表格解析工具，实现 PDF、Word、Excel 等常见文档解析；参与设计 Redis 窗口缓存、Milvus 长期存储、大模型摘要结合的三级记忆方案，优化多轮对话场景。
- 4.使用 JMeter 模拟普通并发场景做压测，配合团队调整异步调度逻辑、修改线程池参数，完成基础的系统性能调优。

技术难点与解决方案:

- 1.难点：单独使用向量检索时，部分关键词匹配不到位，检索结果贴合度一般。
解决方案：搭配 Elasticsearch 做关键词检索，采用双路召回方式，再通过微调后的重排序模型筛选结果，改善检索质量。
- 2.难点：长时间多轮对话时，大模型输入 Token 数量较多，调用成本偏高。

解决方案：采用分层记忆方案，短期对话内容缓存至 Redis，历史知识存入向量库，同时用大模型精简对话摘要，减少单次输入 Token 量。

3.难点：项目部署在 2 核 4G 低配环境，并发请求下偶尔出现响应变慢的情况。

解决方案：优化异步执行逻辑，合理调整线程池参数，减轻单服务运行压力，提升运行稳定性。

落地成果：

- 1.双路检索 + 重排序的方案，相比单一向量检索，检索结果贴合度有明显改善，文档问答体验变好。
- 2.三级记忆方案有效减少了大模型 Token 消耗，平台可以正常支撑较长时长的多轮对话。
- 3.经过压测和参数调优后，系统在 2 核 4G 环境下可稳定运行，普通并发场景下请求成功率、响应延迟均满足基础使用要求。
- 4.平台 Agent 可独立完成大部分基础办公任务，初步实现文档问答和内部系统简易操作联动。

2026.02-2026.06

智能客服与工单自动处理系统

核心开发者

项目背景:企业人工客服工作量大，传统工单流转速度慢、处理不及时。本项目主打多 Agent 协作能力，搭建智能客服与工单自动化系统，实现用户咨询识别、多轮对话、工单创建、进度查询等功能，减轻人工客服压力。

技术栈：Spring Boot 3、LangChain4j、Redis、RabbitMQ、DeepSeek、Vue、WebSocket、JMeter、Docker

工作内容：

- 1.参与设计调度 Agent、业务 Agent、质检 Agent 三级协作架构，协助完成意图识别分发、业务接口调用、结果校验兜底等全流程逻辑开发。
- 2.基于 LangChain4j 结合 Few-shot 提示词，实现 12 类常见客服咨询意图分类；搭建对话状态机，补充多轮对话槽位填充、主动反问的基础能力。
- 3.对接工单系统、用户中心、消息通知三类内部接口，实现工单新建、进度查询、消息催办等功能；借助 RabbitMQ 搭建重试队列，保障异步任务正常执行。
- 4.基于 WebSocket 实现流式对话输出，优化对话响应速度；使用 JMeter 完成基础性能测试，验证接口稳定性。

技术难点与解决方案：

- 1.难点：单一 Agent 同时处理识别、执行、校验多项工作，流程混乱，部分任务执行失败。
解决方案：拆分出三类不同职责的 Agent，分工完成任务分发、业务执行、结果检查，明确各模块边界，降低出错概率。
- 2.难点：用户咨询口语化较强，简单规则无法准确区分咨询意图，多轮对话容易缺失关键信息。
解决方案：使用 Few-shot 提示词提升意图识别准确度，搭配对话状态机，针对缺失信息主动向用户确认，补齐对话槽位。
- 3.难点：外部接口偶尔出现调用超时、网络波动，导致工单创建、消息推送等任务中断。
解决方案：利用 RabbitMQ 搭建重试队列，针对调用失败的任务进行重试，保障异步业务可以正常收尾。
- 4.难点：传统接口请求方式对话输出慢，用户使用体验不佳。
解决方案：改用 WebSocket 长连接实现流式输出，逐步推送对话内容，缩短首条消息等待时间。

落地成果：

- 1.三级多 Agent 协作架构平稳运行，大部分咨询工单可自动处理，异常任务占比控制在合理范围。
- 2.意图分类功能可识别日常 12 类客服问题，多轮对话信息补齐能力有效减少人工客服介入次数。
- 3.消息队列重试机制生效，接口异步调用稳定性提升，工单相关核心流程可正常运转。
- 4.流式对话功能上线后，对话首字响应速度有所加快，用户咨询体验得到改善。